



KPLABS Course

Docker Certified Associate 2020

Getting Started with Dockers

ISSUED BY

Zeal Vora

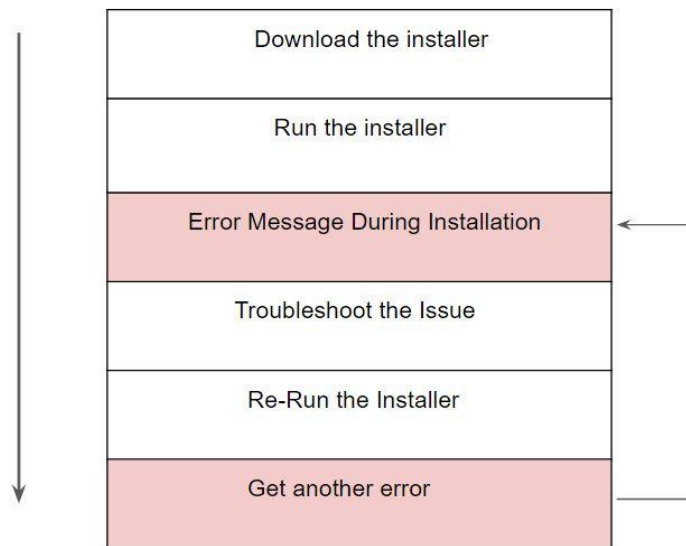
REPRESENTATIVE

instructors@kplabs.in

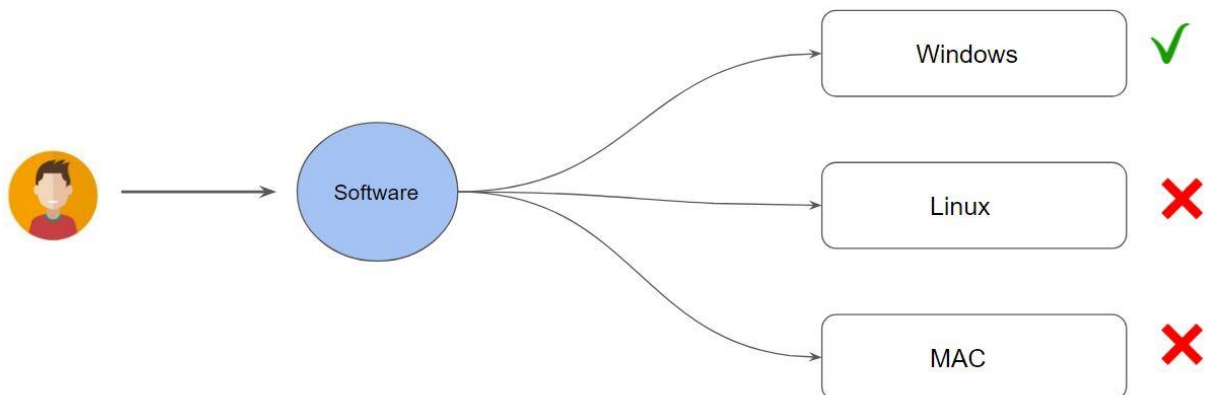
Module 1: Introduction to Docker

1.1 Understanding the Challenge

Every software has its own set of pre-requisite dependencies which must be present before the installation. This leads to many sets of challenges depending on the operating system used.



Along with the above challenge, the second primary issue is related to OS compatibility. A software written for Windows might not work for Linux and MAC and so on.



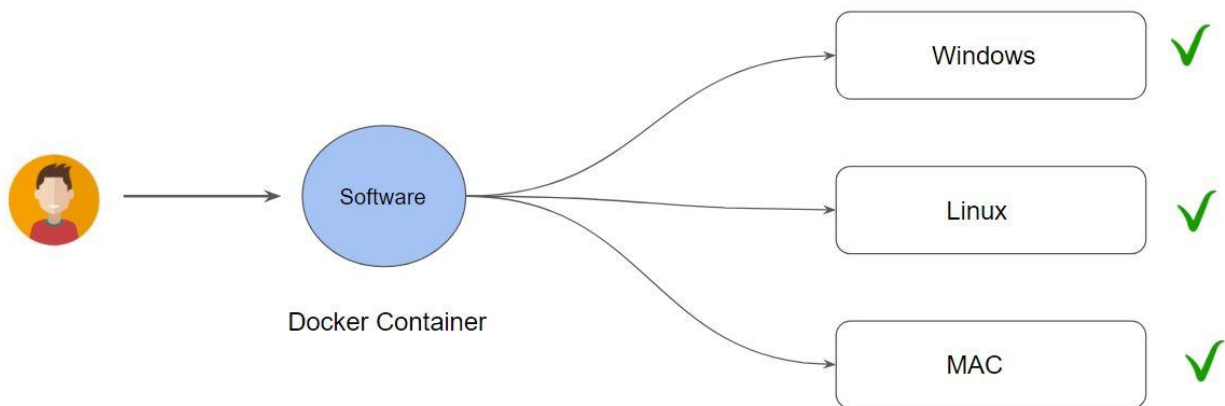
These are some of the primary challenges which Docker is trying to solve.

1.2 Introduction to Docker

Docker is a technology designed to make it easier to create, deploy, and run applications by using containers.

Docker is an open platform, once we build a docker container, we can run it anywhere, say it windows, Linux, mac whether on a laptop, data center, or in the cloud.

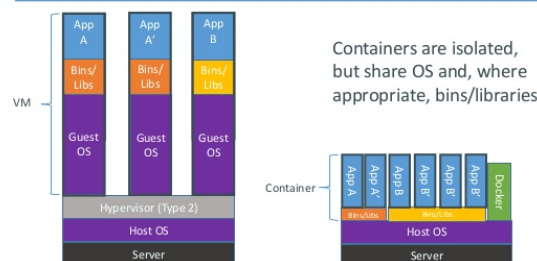
It follows the build once, run anywhere approach.



1.3 Docker Containers vs Virtual Machines

- Virtual Machine contains the entire Operating System.
- The container uses the resource of the host operating system

Containers vs. VMs



Module 2: Installing Docker

Docker works on a wide variety of operating systems, this includes:

- Windows
- Linux
- MAC

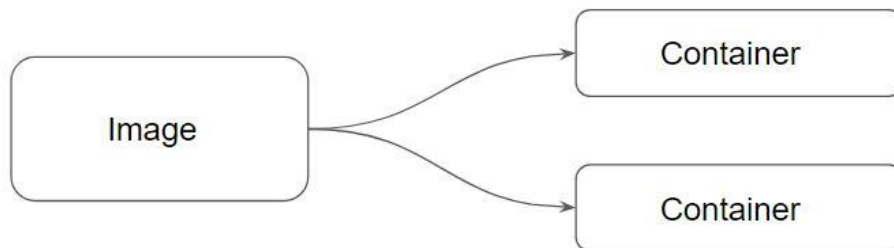
The installation of Docker is pretty straight forward in each one of them.

<https://docs.docker.com/get-docker/>

Module 3: Docker [Image vs Containers]

Docker Image is a file that contains all the necessary dependency and configurations which are required to run an application.

Docker Containers is basically a running instance of an image.



Module 4: Container Identification

When you create a Docker container, it is assigned a universally unique identifier (UUID).

These can help identify the docker container among others.

```
[root@docker-demo ~]# docker run -dt -p 80:80 nginx  
d5187cb1c7f4380b3e37e0c0c811a437d7b8a49d5beb705711a4e54e99d72d77
```

To help humans, Docker also allows us to supply container names.

By default, if we do not specify the name, docker supplies a randomly-generated name from two words, joined by an underscore

```
[root@docker-demo ~]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
d5187cb1c7f4	nginx	"nginx -g 'daemon ..."	47 minutes ago	Up 31 minutes
0.0.0.0:80->80/tcp	inspiring_poitras			

By adding **--name=meaningful_name** argument during the docker run command, we can specify our own name to the containers.

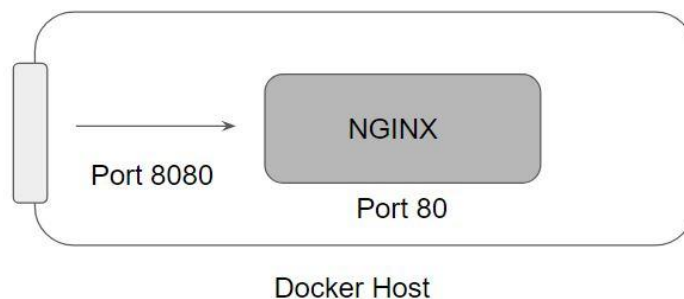
```
[root@docker-demo ~]# docker run --name mynginx -dt -p 800:80 nginx  
9fba1f62038d96159630bd436532ce56105396a6465c1335e16d4d09336cd969  
[root@docker-demo ~]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
9fba1f62038d	nginx	"nginx -g 'daemon ..."	5 seconds ago	Up 5 seconds
0.0.0.0:800->80/tcp	mynginx			

Module 5: Port Binding

By default Docker containers can make connections to the outside world, but the outside world cannot connect to containers.

If we want containers to accept incoming connections from the world, you will have to bind it to a host port.



```
[root@docker-demo ~]# docker run --name mynginx -dt -p 800:80 nginx
9fba1f62038d96159630bd436532ce56105396a6465c1335e16d4d09336cd969
[root@docker-demo ~]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
9fba1f62038d	nginx	"nginx -g 'daemon ...'"	5 seconds ago	Up 5 seconds
0.0.0.0:800->80/tcp	mynginx			

Module 6: Attached and Detached Mode

When we start a docker container, we need to decide if we want to run in a default foreground mode or the detached mode.

You may want to use this if you want a container to run but do not want to view and follow all its output.

Module 7: Removing Docker Containers

Docker containers can be removed with the help of **docker container rm** command.

Description	Command
Remove single container	docker container rm CONTAINER
Stop all the containers	docker container stop \$(docker container ls -aq)
Remove all the containers	docker container rm \$(docker container ls -aq)

Module 8: New Docker CLI Commands

Prior to docker 1.13, the docker run command was only available.

The cli commands were then refactored to have the form docker COMMAND SUBCOMMAND, wherein this case the COMMAND is container and the SUBCOMMAND is run

Older Approach: docker run

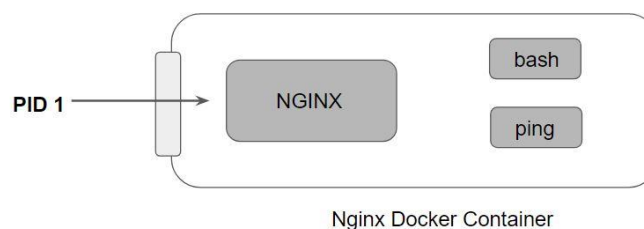
Newer Approach: docker container run

Both of these approaches will work perfectly.

Module 9: docker container exec

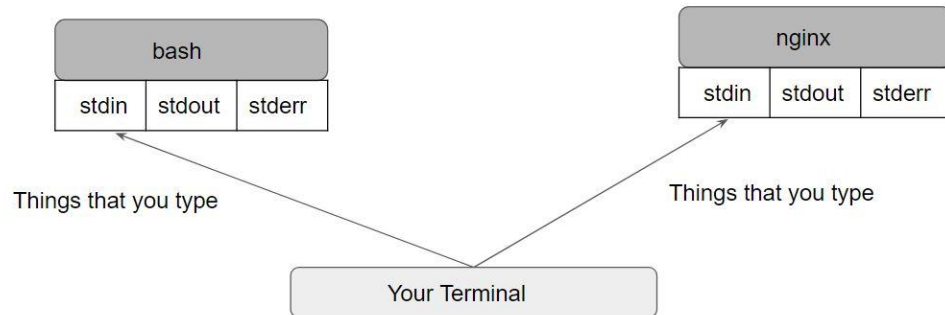
The docker container exec command runs a new command in a running container.

The command started using docker exec only runs while the container's primary process (PID 1) is running, and it is not restarted if the container is restarted.

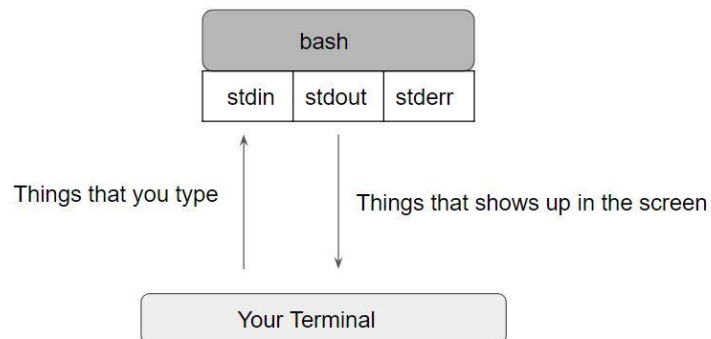


Module 10: Importance of IT Flag

Every process that we create in the Linux environment, has three open file descriptors; stdin, stdout, stderr.



The following diagram shows the difference between STDIN and STDOUT for a specific process (bash)



Following are the two important flags with respect to containers:

--interactive flag keeps stdin open even if not attached.

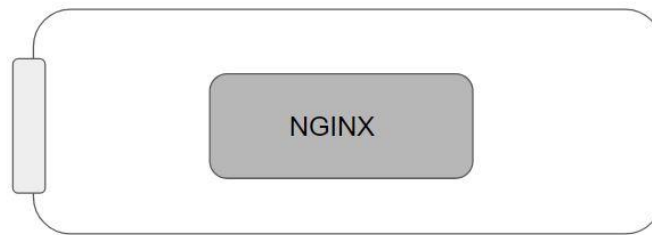
--tty flag allocates a pseudo-TTY

```
[root@docker-demo ~]# docker container exec -it docker-exec bash
root@1cbc45686152:/# netstat -ntlp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      1/nginx: master pro
```

Module 11: Default Container Command

Whenever we run a container, a default command executes which typically runs as PID 1.

This command can be defined while we are defining the container image.



Nginx Container

We can override the default container command by manually specifying the command.

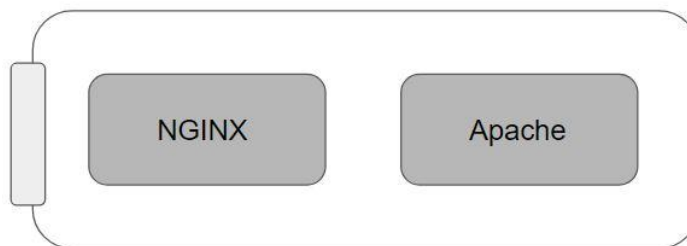
```
docker container run -d nginx sleep 500
```

In the above command, the sleep 500 will run as a PID 1 process overriding any default command that would be present in the nginx container.

Module 12: Restart Policies

By default, Docker containers will not start when they exit or when docker daemon is restarted.

Docker provides restart policies to control whether your containers start automatically when they exit, or when Docker restarts.



Docker Host

We can specify the restart policy by using the `--restart` flag with docker run command.

Flag	Description
no	Do not automatically restart the container. (the default)
on-failure	Restart the container if it exits due to an error, which manifests as a non-zero exit code.
unless-stopped	Restart the container unless it is explicitly stopped or Docker itself is stopped or restarted.
always	Always restart the container if it stops.

Module 13: Disk Usage Metrics

The docker system df command displays information regarding the amount of disk space used by the docker daemon.

```
C:\Users\Zeal Vora>docker system df
```

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	15	1	1.237GB	1.074GB (86%)
Containers	2	0	1.534GB	1.534GB (100%)
Local Volumes	0	0	0B	0B
Build Cache	0	0	0B	0B

Module 14: Automatically Delete Container On Exit

By default, containers that are exited are not removed by Docker.

With the **--rm** flag, the user can specify that whenever a container exits, it should automatically be removed.